

Objectives

- Explain various ways of conditional rendering
- Explain how to render multiple components
- Define list component
- Explain about keys in React applications
- Explain how to extract components with keys
- Explain React Map, map() function

In this hands-on lab, you will learn how to:

- Implement conditional rendering in React applications

Prerequisites

The following is required to complete this hands-on lab:

- Node.js
- NPM
- Visual Studio Code

Notes

Estimated time to complete this lab: **60 minutes**.

Create a React App named “bloggerapp” in with 3 components.

1. Book Details
2. Blog Details
3. Course Details

Implement this with as many ways possible of Conditional Rendering.



Conditional rendering means displaying different UI based on a condition.

a) Using if...else

```
function Login(props) {  
  if (props.isLoggedIn) {  
    return <h1>Welcome User</h1>;  
  } else {  
    return <h1>Please Login</h1>;  
  }  
}
```

b) Using Ternary Operator (? :)

```
function Login(props) {  
  return (  
    <h1>  
      {props.isLoggedIn ? "Welcome User" : "Please Login"}  
    </h1>  
  );  
}
```

c) Using Logical AND (&&)

Used when only one condition needs to display something.

```
function Message(props) {  
  return (  
    <div>  
      {props.hasMessage && <h2>You have new messages.</h2>}  
    </div>  
  );  
}
```

d) Using Element Variables

```
function Greeting(props) {  
  let message;  
  
  if (props.isLoggedIn) {  
    message = <h1>Welcome</h1>;  
  } else {  
    message = <h1>Please Login</h1>;  
  }  
  
  return <div>{message}</div>;  
}
```

2. Rendering Multiple Components

A parent component can render multiple child components.

Example:

```
function Header() {  
  return <h1>Header</h1>;  
}
```

```
function Body() {
```

```
    return <p>This is body.</p>;  
  }
```

```
function Footer() {  
  return <h3>Footer</h3>;  
}
```

```
function App() {  
  return (  
    <div>  
      <Header />  
      <Body />  
      <Footer />  
    </div>  
  );  
}
```

```
export default App;
```

List Component

A list component displays multiple items using an array.

4. Keys in React Applications

A **key** is a unique identifier given to list elements.

It helps React:

- Identify changed items
- Update the UI efficiently
- Improve performance

Example:

```
const fruits = ["Apple", "Orange", "Mango"];
```

```
function FruitList() {  
  return (  
    <ul>  
      {fruits.map((fruit, index) => (  
        <li key={index}>{fruit}</li>  
      ))}  
    </ul>  
  );  
}
```

5. Extracting Components with Keys

Instead of writing everything in one component, create a separate component.

Example:

```
function Student(props) {  
  return <li>{props.name}</li>;  
}
```

```
const students = [  
  { id: 1, name: "John" },  
  { id: 2, name: "Sam" }  
];
```

```
function StudentList() {  
  return (  
    <ul>  
      {students.map(student => (  
        <Student
```

```
        key={student.id}
        name={student.name}
      />
    )}]
  </ul>
);
}
```

6. React map() Function

map() is a JavaScript array method used to transform each element into a React element.

Syntax:

```
array.map((item) => {
  return JSX;
});
```